

LC 140: WORD BREAK - II & LC 139

Authored by: Arpit Raj, LNMIIT Jaipur | Date: August 30, 2026

Asked in: **Dropbox, Zillow, Adobe, Pocket, Mahindra, Mobisoft, Capgemini**

Problem Statement

Given a string **s** and a dictionary of strings, need to add partition/space in such a way that each word is a valid dictionary word. Need to return all such sentences in order.

Example Walkthrough

String (S)	aadyaiamobsessed
Dictionary	aadya, ai, obsessed, mob, i, am, aady, a, sessed
Output Sentences	1. aady a i a mob sessed 2. aadya i a mob sessed 3. aadya ai a mob sessed 4. aady a i am obsessed 5. aadya i am obsessed 6. aady ai am obsessed

Partition DP Approach (LC 139 & LC 140)

See this as a partition DP problem:

`[0 ..... j-1]` (already solved) | `[j ..... i-1]` (new word)

For LC 139 (Word Break I):

If `s[j.....i-1]` is present in dict and `dp[j]` was true, then `dp[i]` is true.

To solve LC 139 Word Break - I, we just need to tell if a string can be segmented or not, so simply return `dp[n]`.

Now similarly here too, ill use partition DP for LC 140:

- `dp[i]` → will store all possible sentences that can be made from first `i` characters
- `dp[0].push_back("");`
- Then `[0.....j-1] [j.....i-1]`

`if(dp[j].empty()) continue;`

(kyunki agar previous part se koi word nahi ban rha to useless)

Iterating over previous states:

```
for (auto &prev : dp[j]) {
    if (prev == "") {
        dp[i].push_back(word);
    }
    else {
        dp[i].push_back(prev + " " + word);
    }
}
```

C++ Implementation

```
class Solution {
public:
    vector<string> wordBreak(string s, vector<string>& wordDict) {
        int n=s.size();
        unordered_set<string> st;

        //for o(1) fast lookup
        for(auto &wrd:wordDict){
            st.insert(wrd);
        }

        vector<vector<string>> dp(n+1);
        //dp[i] will store sentences that can be made from first i characters

        dp[0].push_back("");

        for(int i=1;i<=n;i++){
            for(int j=i-1;j>=0;j--){

                // [0.....j-1][j.....i-1];

                //agar first j character se word nhi ban rha , to partition ka sense nahi
                if(dp[j].empty()) continue;

                string wrd=s.substr(j,i-j);
                if(st.find(wrd)!=st.end()){

                    for(auto &prev:dp[j]){
                        if(prev==""){
                            dp[i].push_back(wrd);
                        }
                        else{
                            dp[i].push_back(prev+" "+wrd);
                        }
                    }
                }
            }
        }

        return dp[n];
    }
};
```

